Low-Level Design: Working Through a Problem

8 min read Updated 28 Jul 2026 Topic 65 of 65

Bookmark Mark done

Why it matters

The LLD round asks you to design something concrete — a parking lot, a rate limiter, a ride matcher — in 45 minutes, out loud. It is not testing whether you know the "right" answer; it is testing whether you can decompose a vague problem, justify trade-offs, and produce something a team could build. Candidates fail it by jumping to classes before agreeing what the system does.

The concept, simply

A repeatable structure that works for almost any prompt:

1. **Clarify and scope** (5 min) — ask questions, state what is in and out. Nobody can design "a parking lot" without knowing whether it handles payments.
2. **Identify entities and behaviours** (5 min) — the nouns, and what they do.
3. **Define the seams** (10 min) — the interfaces where behaviour will vary.
4. **Write the core classes** (15 min) — real Java, not boxes and arrows.
5. **Discuss trade-offs** (10 min) — concurrency, extension, what changes at scale.

Step 1 is the one people skip and the one marked hardest. Designing the wrong system elegantly scores worse than designing the right one plainly.

Code examples

A worked parking lot — the canonical prompt:

```java
// 1. SCOPE — say this out loud before writing anything:
//    In:  multiple levels, vehicle sizes, park/unpark, find a free spot,
//         hourly pricing. Out: payments, reservations, plate recognition.
//    Assume one JVM, concurrent access from several entry gates.

// 2. ENTITIES — records for the values (Chapter 10)
enum VehicleSize { MOTORCYCLE, CAR, VAN }
record Vehicle(String plate, VehicleSize size) { }
record Ticket(String id, String plate, String spotId, Instant entryTime) {

// 3. SEAMS — the two things most likely to change
interface SpotAllocationStrategy { Optional<Spot> allocate(Deque<Spot> fre
interface PricingStrategy { BigDecimal price(Duration parked, VehicleSize

// 4. CORE
public class ParkingLot {
    private final Map<VehicleSize, Deque<Spot>> freeBySize;   // O(1) allo
    private final Map<String, Ticket> active = new ConcurrentHashMap<>();
    private final SpotAllocationStrategy allocation;
    private final PricingStrategy pricing;
    private final Lock lock = new ReentrantLock();

    public Ticket park(Vehicle vehicle) {
        lock.lock();                          // ✓ allocation must be atomic
        try {                                 //   gates must not get the sam
            Spot spot = allocation.allocate(freeBySize.get(vehicle.size())
                    .orElseThrow(() -> new LotFullException(vehicle.size()
            spot.occupy(vehicle);
            Ticket t = new Ticket(UUID.randomUUID().toString(), vehicle.pl
                                  spot.id(), Instant.now());
            active.put(t.id(), t);
            return t;
        } finally { lock.unlock(); }       // ✓ Chapter 8's rule
    }

    public BigDecimal unpark(String ticketId) {
        Ticket t = active.remove(ticketId);
        if (t == null) throw new UnknownTicketException(ticketId);
        release(t.spotId());
        return pricing.price(Duration.between(t.entryTime(), Instant.now()
```

Then say the trade-offs out loud — this is where the marks actually are:

TEXT Copy

```
"A Deque of free spots per size makes allocation O(1) instead of scanning
 every spot. I hold one lock across allocation because two gates must never
 be handed the same spot; at higher scale I would lock per level, or use a
 compare-and-set on each spot to remove the global lock entirely.

 Allocation and pricing are interfaces because those are the two things a
 customer will want to change - nearest-to-lift versus first-available,
 flat rate versus tiered. Everything else is stable, so I have not
 abstracted it.

 Across multiple JVMs the in-memory maps stop working. Spot state moves to
 the database and allocation becomes a conditional UPDATE, which gives
 atomicity without a distributed lock."
```

A second, smaller worked example — a rate limiter, which comes up constantly:

JAVA Copy

```java
// Token bucket: N tokens, refilled at a fixed rate, one consumed per requ
// Chosen over a fixed window because a fixed window allows a burst of 2N
// across a boundary - the classic follow-up question.
public class TokenBucket {
    private final long capacity;
    private final double refillPerSecond;
    private double tokens;
    private long lastRefillNanos;

    public synchronized boolean tryConsume() {
        refill();
```

```
        if (tokens < 1) return false;
        tokens -= 1;
        return true;
    }

    private void refill() {                          // lazy refill — no time
        long now = System.nanoTime();
        double elapsed = (now - lastRefillNanos) / 1_000_000_000.0;
        tokens = Math.min(capacity, tokens + elapsed * refillPerSecond);
        lastRefillNanos = now;
    }
}
// Trade-offs to state: per-user buckets need a ConcurrentHashMap with
// eviction or it leaks (Chapter 9); distributed rate limiting needs Redis
// with an atomic script, because per-instance buckets multiply the limit
// the number of instances.
```

Under the hood

What is actually being assessed is rarely the final class diagram. It is whether you asked clarifying questions, whether your abstractions sit where change is likely, whether you noticed the concurrency, and whether you could explain a decision you did *not* take. Interviewers routinely say the strongest signal is a candidate saying "I would not abstract that yet, because I have only one case" — it demonstrates judgement rather than pattern application.

Common prompts and the concept each is testing: parking lot (allocation and concurrency), elevator (state machines and scheduling), rate limiter (algorithms and distribution), cache with eviction (data structures — Chapter 4's `LinkedHashMap` LRU), ride matching (spatial indexing and concurrency), splitwise or a vending machine (state and money handling — `BigDecimal`, never `double`).

The habits that carry across all of them: put money in `BigDecimal` or minor units, never `double` (Chapter 1); use immutable records for values and classes for things with a lifecycle; make illegal states unrepresentable rather than validating them everywhere; and say what you would change at ten times the scale, because that question is coming.

Common mistakes

- **Coding before scoping.** The first five minutes are questions, not classes.
- **Abstracting everything.** Two well-chosen interfaces beat eight speculative ones.
- **Ignoring concurrency.** Almost every prompt has concurrent actors; say how you handle it.
- `double` **for money.** An immediate red flag (Chapter 1).
- **Designing in silence.** The reasoning is the answer; the diagram is a by-product.
- **No trade-off discussion.** Every choice excludes another — name it.
- **Unbounded in-memory maps.** A per-user map with no eviction is a leak (Chapter 9).

Interview lens

> **Q: "Design a parking lot."**
> A: I would scope first — multiple levels and vehicle sizes, park and unpark, hourly pricing; payments and reservations out. Entities: `Vehicle` and `Ticket` as records, `Spot` as a class because it has state. The two seams I would abstract are allocation and pricing, since those are what a customer changes. I would keep free spots in a deque per size for O(1) allocation, hold a lock across allocation so two gates cannot get the same spot, and note that at scale I would move that to per-level locking or a compare-and-set on each spot.

> **Q: "How do you decide what to abstract?"**
> A: By asking what is likely to change. In a parking lot, allocation strategy and pricing will change; the fact that a ticket has an entry time will not. I would rather have two well-chosen interfaces than eight speculative ones, and I will say out loud when I am deliberately *not* abstracting something because there is only one case today.

> **Q: "Design a rate limiter."**
> A: Token bucket — a capacity of N tokens refilled at a fixed rate, one consumed per request — refilled lazily on each call so there is no timer thread. I prefer it to a fixed window because a fixed window lets a client send 2N requests across a boundary. Per-user buckets go in a `ConcurrentHashMap` with eviction, since an unbounded

one is a memory leak. For multiple instances, per-instance buckets multiply the limit by the instance count, so it moves to Redis with an atomic script.

Q: "What are they really assessing?"

A: Not the class diagram. Whether I clarified the problem before solving it, whether my abstractions sit where change is likely, whether I noticed the concurrency, and whether I can justify a decision including the ones I rejected. Saying "I would not abstract that yet — I only have one case" is usually a stronger signal than adding another interface.

Q: "How would this change at ten times the scale?"

A: For the parking lot, the in-memory maps stop working across instances, so spot state moves to the database and allocation becomes a conditional `UPDATE` — atomic without a distributed lock. Generally: identify the shared mutable state, decide whether it moves to a database or a cache, replace in-process locks with database-level atomicity or optimistic locking, and check that nothing unbounded grows per user.

Cheat-card

- Structure: **scope → entities → seams → code → trade-offs**. Never skip scoping.
- Records for values, classes for things with a **lifecycle**.
- Abstract only **where change is likely** — and say when you deliberately do not.
- **Name the concurrency**: what is shared, what lock, what you would do at scale.
- Money = `BigDecimal` **or minor units**, never `double`.
- Per-user maps need **eviction** or they leak.
- Common prompts: parking lot · elevator · rate limiter · LRU cache · vending machine.
- **Think out loud** — the reasoning is what is being marked.

QUICK CHECK — no marks, just for you.

- Writing code that does not compile
- Jumping straight to classes without clarifying and scoping the problem first
- Choosing the wrong data structure
- Not knowing enough design patterns

Check answer

2. In a parking lot design, which parts are worth putting behind an interface?

- The Ticket and Vehicle types
- Only the persistence layer
- Spot allocation and pricing — the two things a customer will actually want to change
- Every class, so the design is fully flexible

Check answer

3. Why is a token bucket usually preferred over a fixed window for rate limiting?

- It does not require a timer thread
- It is easier to distribute across instances
- It uses less memory
- A fixed window allows a burst of 2N requests across a window boundary; a token bucket smooths the rate continuously

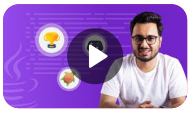
Previous
Design Patterns That Still Earn Their Keep

LEARN MORE

ARTICLES & DOCS

- Concurrency — The Java Tutorials
  docs.oracle.com
- BigDecimal — API docs
  docs.oracle.com

VIDEOS



Low Level Design Course - Master Design Patterns & SOLID Principles using Java
Prateek Narang



SOLID principles explained | Java | System Design Interview
ByteMonk

Hand-picked references for this lesson.